



Pandas & SQL: Data Retrieval

Nguyễn Thế Hào và Đinh Quang Vinh

I. Lý thuyết



Data Retrieval là gì?

Data Retrieval là bước **lấy đúng dữ liệu cần dùng** từ nguồn dữ liệu (CSV/Database) để phục vụ phân tích và mô hình hoá.

Các thao tác thường gặp:

- **Projection**: chọn cột cần thiết.
- **Selection**: lọc dòng theo điều kiện.
- **Ordering & Top-N**: sắp xếp và lấy N dòng theo tiêu chí.
- **Aggregation**: tổng hợp theo nhóm để tạo bảng báo cáo.
- **Computed fields**: tạo thêm cột tính toán (ví dụ $GDP \approx \text{dân số} \times GDP/\text{người}$).

II. Case study: Gapminder (Simple)

Bộ dữ liệu Gapminder (Simple)

Dataset `gapminder_simple.csv` là phiên bản rút gọn (1000 samples) từ Gapminder. Các cột chính:

- | | |
|---------------------------------|--|
| • country : tên quốc gia | • lifeExp : tuổi thọ trung bình |
| • continent : châu lục | • pop : dân số |
| • year : năm quan sát | • gdpPercap : GDP bình quân đầu người |

Mục tiêu: luyện các thao tác retrieval cơ bản (projection, selection, top-N, aggregation, computed fields) bằng cả Pandas và SQL. Các bạn có thể tải data theo link

[Data](#).

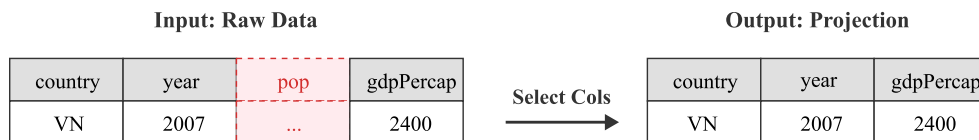
II.1. Import & đọc dataset

Pandas

```
1 import pandas as pd
2
3 df = pd.read_csv("gapminder_simple.csv")
4 df.head()
```

II.2. Bước 1: Projection

Mục tiêu là chỉ giữ các cột phục vụ phân tích (dataset gọn hơn, dễ nhìn hơn).



Pandas

```
1 use_cols = ["country", "continent", "year", "lifeExp", "gdpPercap"]
2 df1 = df[use_cols].copy()
3
4 df1.head(10)
```

Giải thích (Pandas)

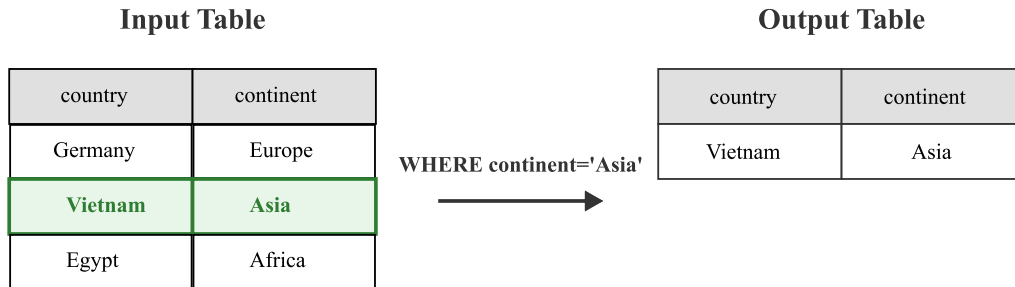
- Projection giúp bỏ các cột chưa cần (ở đây bỏ pop) để tập trung vào mục tiêu hiện tại.
- copy() giúp thao tác an toàn khi bạn muốn tạo thêm cột hoặc chỉnh sửa sau đó.

SQL

```
1 SELECT country, continent, 'year', lifeExp, gdpPercap
2 FROM gapminder_simple
3 LIMIT 10;
```

II.3. Bước 2: Selection

Mục tiêu là lọc theo châu lục và năm để lấy đúng phạm vi cần phân tích.



Pandas

```

1 # Ví dụ: Chỉ lấy Asia, giai đoạn 1990-2007
2 mask = (df["continent"] == "Asia") & (df["year"] >= 1990) & (df["year"]
3   <= 2007)
4 asia_1990_2007 = df.loc[mask, ["country", "continent", "year", "lifeExp", "
5   gdpPercap", "pop"]].copy()
6
7 asia_1990_2007.head(10)

```

Giải thích (Pandas)

- Điều kiện lọc trong Pandas dùng boolean mask, tương đương WHERE trong SQL.
- `df.loc[mask, cols]` vừa lọc dòng vừa chọn cột, rất hay dùng trong retrieval.

SQL

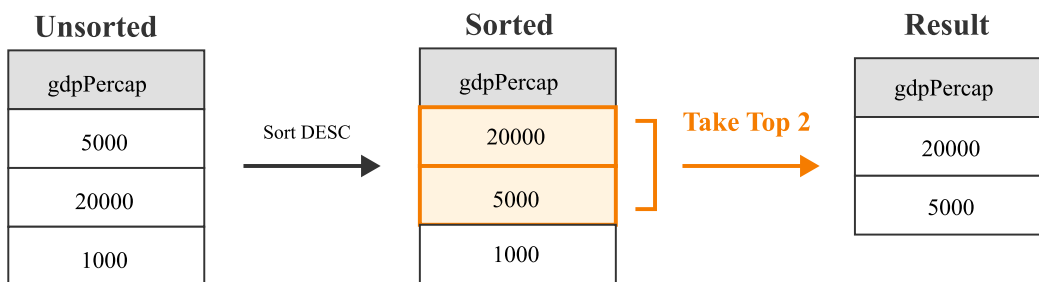
```

1 SELECT country, continent, 'year', lifeExp, gdpPercap, pop
2 FROM gapminder_simple
3 WHERE continent = 'Asia'
4   AND 'year' BETWEEN 1990 AND 2007
5 LIMIT 10;

```

II.4. Bước 3: Ordering & Top-N

Mục tiêu là lấy top 10 quốc gia có gdpPercap cao nhất trong năm 2007.



Pandas

```

1 df_2007 = df[df["year"] == 2007].copy()
2
3 top10_gdpPercap_2007 = (
4     df_2007.sort_values("gdpPercap", ascending=False)
5     [["country", "continent", "year", "gdpPercap", "lifeExp"]]
6     .head(10)
7 )
8
9 top10_gdpPercap_2007

```

Giải thích (Pandas)

- `sort_values(..., ascending=False)` sắp xếp giảm dần theo tiêu chí.
- `head(10)` lấy top 10, tương đương LIMIT 10.

SQL

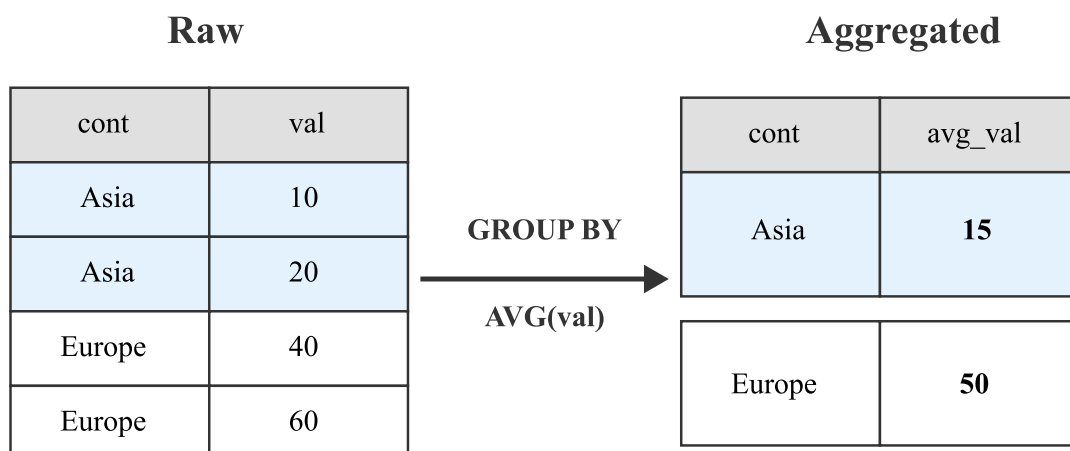
```

1 SELECT country, continent, 'year', gdpPercap, lifeExp
2 FROM gapminder_simple
3 WHERE 'year' = 2007
4 ORDER BY gdpPercap DESC
5 LIMIT 10;

```

II.5. Bước 4: Aggregation đơn giản

Mục tiêu: tổng hợp theo châu lục trong năm 2007 với 2 chỉ số cơ bản: **tuổi thọ trung bình** và **GDP/người trung bình**.



Pandas

```

1 tmp = df[df["year"] == 2007].copy()
2
3 agg_continent_2007 = (
4     tmp.groupby("continent", as_index=False)
5         .agg(
6             avg_lifeExp=("lifeExp", "mean"),
7             avg_gdpPercap=("gdpPercap", "mean")
8         )
9         .sort_values("avg_gdpPercap", ascending=False)
10 )
11
12 agg_continent_2007

```

Giải thích (Pandas)

- `groupby("continent")` chia dữ liệu theo châu lục.
- `mean()` cho ra bảng summary gọn, đúng tinh thần retrieval cho báo cáo nhanh.

SQL

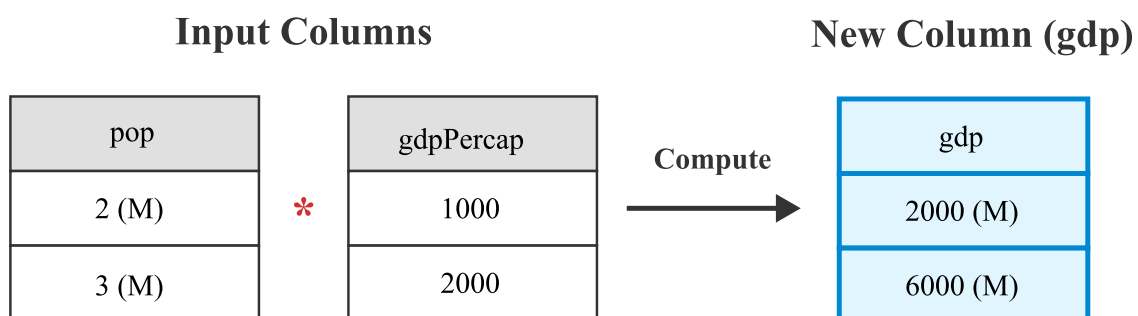
```

1 SELECT
2     continent,
3     AVG(lifeExp) AS avg_lifeExp,
4     AVG(gdpPercap) AS avg_gdpPercap
5 FROM gapminder_simple
6 WHERE 'year' = 2007
7 GROUP BY continent
8 ORDER BY avg_gdpPercap DESC;

```

II.6. Bước 5: Computed fields

Mục tiêu: tạo GDP xấp xỉ theo công thức $GDP = pop * gdpPercap$, rồi lấy top theo GDP trong năm 2007.



Pandas

```
1 tmp = df[df["year"] == 2007].copy()
2 tmp["gdp"] = tmp["pop"] * tmp["gdpPercap"]
3
4 top10_gdp_2007 = (
5     tmp.sort_values("gdp", ascending=False)
6     [["country", "continent", "year", "pop", "gdpPercap", "gdp", "lifeExp"]]
7     .head(10)
8 )
9
10 top10_gdp_2007
```

Giải thích (Pandas)

- `gdp = pop * gdpPercap` là computed field tạo ngay trong bước retrieval.
- Sau khi có cột `gdp`, thao tác `sort` + `head` giống hệt Top-N.

SQL

```
1 SELECT
2     country, continent, 'year',
3     pop, gdpPercap,
4     (pop * gdpPercap) AS gdp,
5     lifeExp
6 FROM gapminder_simple
7 WHERE 'year' = 2007
8 ORDER BY gdp DESC
9 LIMIT 10;
```



Pandas & SQL - Data Aggregation

Nguyễn Thế Hào và Đinh Quang Vinh

I. Lý thuyết

Data Aggregation & Calculation

Data Aggregation là gom nhóm dữ liệu theo một hoặc nhiều khóa và tính các thống kê tóm tắt cho từng nhóm. **Calculation** là tạo biến dẫn xuất (computed field) để phục vụ phân tích và tổng hợp.

Đặc điểm chính:

- Kết quả thường **giảm số dòng**: mỗi nhóm tương ứng một dòng tổng hợp.
- Thống kê phổ biến: count, sum, average, min, max.
- **WHERE** lọc trước khi gom nhóm, **HAVING** lọc sau khi đã tổng hợp.
- Có thể tổng hợp trực tiếp trên biến mới (vd. $gdp = pop \times gdpPercap$).

II. Case study

Bộ dữ liệu Gapminder (Simple)

gapminder_simple.csv (1000 mẫu) là bản rút gọn từ Gapminder, mô tả chỉ số phát triển theo quốc gia và năm.

- **Khóa & phân nhóm**: country, year, continent.
- **Biến số**: lifeExp, pop, gdpPercap; tạo $gdp = pop \times gdpPercap$ để tính GDP tổng.

Các bước thực hành (keywords): computed field → overall summary → groupby(continent) → groupby(year) → filter+TopN(2007) → groupby(continent,year).

Dataset (CSV): [link](#)

II.1. Bước 0 (Pandas): Import và đọc dataset

Pandas

```
1 import pandas as pd
2
3 DATA_PATH = "/content/gapminder_simple.csv"
4 df = pd.read_csv(DATA_PATH)
```

Giải thích

Chỉ đọc dữ liệu vào DataFrame để chuẩn bị cho các bước tổng hợp. (Không preview dữ liệu.)

II.2. Bước 1: Tạo computed field gdp

Mục tiêu: tạo biến mới $gdp = pop * gdpPercap$ để dùng trong các phép SUM theo nhóm.

Pandas

```
1 df["gdp"] = df["pop"] * df["gdpPercap"]
```

SQL

```
1 SELECT pop * gdpPercap AS gdp
2 FROM gapminder_simple;
```

Giải thích

Computed field giúp minh họa đúng tinh thần “calculation rồi mới aggregation”.

II.3. Bước 2: Thống kê tổng quan (overall summary)

Mục tiêu: tóm tắt nhanh các cột số quan trọng bằng COUNT/AVG/MIN/MAX.

Pandas

```
1 overall = df[["lifeExp", "gdpPercap", "pop"]].agg(["count", "mean", "min",
2 overall
, "max"])
```


SQL

```

1 SELECT
2     COUNT(*)           AS n,
3     AVG(lifeExp)       AS avg_lifeExp,
4     AVG(gdpPercap)    AS avg_gdpPercap
5 FROM gapminder_simple;

```

Giải thích

Bước này không nhìn từng dòng mà nhìn **thống kê tổng quát** để hiểu thang đo dữ liệu.

II.4. Bước 3: Tổng hợp theo châu lục (continent)

Mục tiêu: báo cáo theo continent: số quốc gia, dân số tổng, tuổi thọ trung bình.

Pandas

```

1 by_continent = (
2     df.groupby("continent", as_index=False)
3     .agg(
4         n_countries=("country", "nunique"),
5         sum_pop=("pop", "sum"),
6         avg_lifeExp=("lifeExp", "mean")
7     )
8     .sort_values("sum_pop", ascending=False)
9 )
10 by_continent

```

SQL

```

1 SELECT
2     continent,
3     COUNT(DISTINCT country) AS n_countries,
4     SUM(pop)                AS sum_pop,
5     AVG(lifeExp)            AS avg_lifeExp
6 FROM gapminder_simple
7 GROUP BY continent
8 ORDER BY sum_pop DESC;

```

Giải thích

GROUP BY continent (SQL) tương đương groupby("continent") (Pandas): mỗi châu lục ra 1 dòng tổng hợp.

II.5. Bước 4: Xu hướng theo năm (year)

Mục tiêu: xem xu hướng theo thời gian với AVG(lifeExp) và AVG(gdpPercap).

Pandas

```
1 by_year = (  
2     df.groupby("year", as_index=False)  
3         .agg(  
4             avg_lifeExp=("lifeExp", "mean"),  
5             avg_gdpPercap=("gdpPercap", "mean")  
6         )  
7     .sort_values("year")  
8 )  
9 by_year
```

SQL

```
1 SELECT  
2     year,  
3     AVG(lifeExp) AS avg_lifeExp,  
4     AVG(gdpPercap) AS avg_gdpPercap  
5 FROM gapminder_simple  
6 GROUP BY year  
7 ORDER BY year;
```

Giải thích

Aggregation theo year tạo báo cáo dạng time trend.

II.6. Bước 5 (đơn giản): Top-N châu lục theo GDP tổng năm 2007

Mục tiêu: lọc year=2007, tính sum_gdp theo châu lục, lấy top 5.

Pandas

```
1 top5_cont_2007 = (  
2     df[df["year"] == 2007]  
3         .groupby("continent", as_index=False)  
4         .agg(sum_gdp=("gdp", "sum"))  
5         .sort_values("sum_gdp", ascending=False)  
6         .head(5)  
7 )  
8 top5_cont_2007
```

SQL

```

1 SELECT
2     continent,
3     SUM(pop * gdpPercap) AS sum_gdp
4 FROM gapminder_simple
5 WHERE year = 2007
6 GROUP BY continent
7 ORDER BY sum_gdp DESC
8 LIMIT 5;

```

Giải thích

Kết hợp filter + groupby + sum + top-N. SQL tương ứng: WHERE + GROUP BY + ORDER BY + LIMIT.

II.7. Bước 6: GDP tổng theo châu lục và năm (continent, year)

Mục tiêu: tạo bảng 2 chiều theo continent, year để so sánh theo thời gian trong từng châu lục.

Pandas

```

1 gdp_by_cont_year = (
2     df.groupby(["continent", "year"], as_index=False)
3     .agg(sum_gdp=("gdp", "sum"))
4     .sort_values(["year", "sum_gdp"], ascending=[True, False])
5 )
6 gdp_by_cont_year

```

SQL

```

1 SELECT
2     continent, year,
3     SUM(pop * gdpPercap) AS sum_gdp
4 FROM gapminder_simple
5 GROUP BY continent, year
6 ORDER BY year, sum_gdp DESC;

```

Giải thích

GROUP BY continent, year giúp so sánh giữa các châu lục theo từng năm.



Pandas & SQL - Data Cleaning

Nguyễn Thế Hào và Đinh Quang Vinh

I. Lý thuyết



Data Cleaning & Preparation

Data Cleaning & Preparation là quá trình chuyển dữ liệu thô thành dữ liệu sẵn sàng cho phân tích và mô hình hóa. Nội dung thường bao gồm:

- Nhận diện và xử lý **giá trị thiếu** (NULL/NaN/blank/sentinel)
- Chuẩn hóa **kiểu dữ liệu** (number, datetime, boolean, category)
- Chuẩn hóa **chuỗi và biến phân loại** (trim, chữ hoa/thường, mapping)
- Phát hiện và xử lý **trùng lặp** (trùng dòng, trùng khóa nghiệp vụ)
- Phát hiện **giá trị không hợp lệ** và **giá trị bất thường** (invalid/outlier)
- Kiểm tra kết quả sau làm sạch và tổng hợp báo cáo thay đổi

II. Case study: Air Quality (UCI)

Bộ dữ liệu Air Quality

Chúng ta sử dụng dữ liệu thực từ UCI (Air Quality), ghi theo **mỗi giờ.S**

- **Cột thời gian:** Date (DD/MM/YYYY), Time (HH.MM.SS).
- **Biến đo:** nồng độ khí (CO, NO_x, NO₂, Benzene, ...) và phản hồi cảm biến (PT08.*), kèm nhiệt độ/độ ẩm.
- **Missing:** các giá trị thiếu được gán bằng -200.

Các bạn có thể tải dataset ở đây: [link](#)

II.1. Chuẩn bị: Import và đọc dataset

Mục tiêu là import các thư viện cần thiết và đọc dataset `airquality_uci_simple.csv`. (Trong Google Colab, file thường được upload vào thư mục `/content`.)

Pandas: Import

```
1 import numpy as np
2 import pandas as pd
```

Pandas: Đọc dataset

```
1 DATA_PATH = "/content/airquality_uci_simple.csv"
2 df = pd.read_csv(DATA_PATH)
3
4 df.shape
5 df.head()
```

II.2. Bước 1: Profiling

Mục tiêu là **xem nhanh dữ liệu** và **đếm mức độ thiếu** ở các cột quan trọng. Với Air Quality, dữ liệu thiếu thường được mã hóa bằng **-200**.

CO(GT)	NO2(GT)	T
2.6	113	13.6
-200	92	-200
1.2	88	11.2

Pandas: Profiling nhanh

```
1 use_cols = ["CO(GT)", "NO2(GT)", "T", "RH"]
2
3 (df[use_cols] == -200).sum()
```

Giải thích code (Pandas)

- `use_cols`: chọn trước các cột quan trọng để thao tác gọn và nhất quán ở các bước sau.
- `df[use_cols] == -200`: tạo bảng **True/False** cho từng ô (ô nào bằng -200).
- `.sum()`: Pandas coi **True=1, False=0** $\Rightarrow$ tổng theo cột chính là **đếm số lượng -200** theo từng cột.

SQL: Profiling

```

1 SELECT
2   COUNT(*) n_rows,
3   SUM('CO(GT)'=-200) miss_co,
4   SUM('NO2(GT)'=-200) miss_no2,
5   SUM('T'=-200) miss_t,
6   SUM('RH'=-200) miss_rh
7 FROM air_quality;

```

II.3. Bước 2: Tạo cột thời gian (timestamp)

Mục tiêu là tạo cột **ts** từ Date và Time để thao tác theo thời gian.



Pandas: Parse timestamp

```

1 import pandas as pd
2
3 df["ts"] = pd.to_datetime(
4     df["Date"].astype(str) + " " + df["Time"].astype(str),
5     format="%d/%m/%Y %H.%M.%S",
6     errors="coerce"
7 )
8
9 df[["Date", "Time", "ts"]].head()

```

Giải thích code (Pandas)

- Ghép Date và Time thành chuỗi "dd/mm/YYYY HH.MM.SS" để parse một lần.
- `format="%d/%m/%Y %H.%M.%S"`: chỉ rõ định dạng (lưu ý **giờ dùng dấu chấm**).
- `errors="coerce"`: parse lỗi thì trả về NaT thay vì báo lỗi, giúp bước sau lọc bỏ dễ dàng.

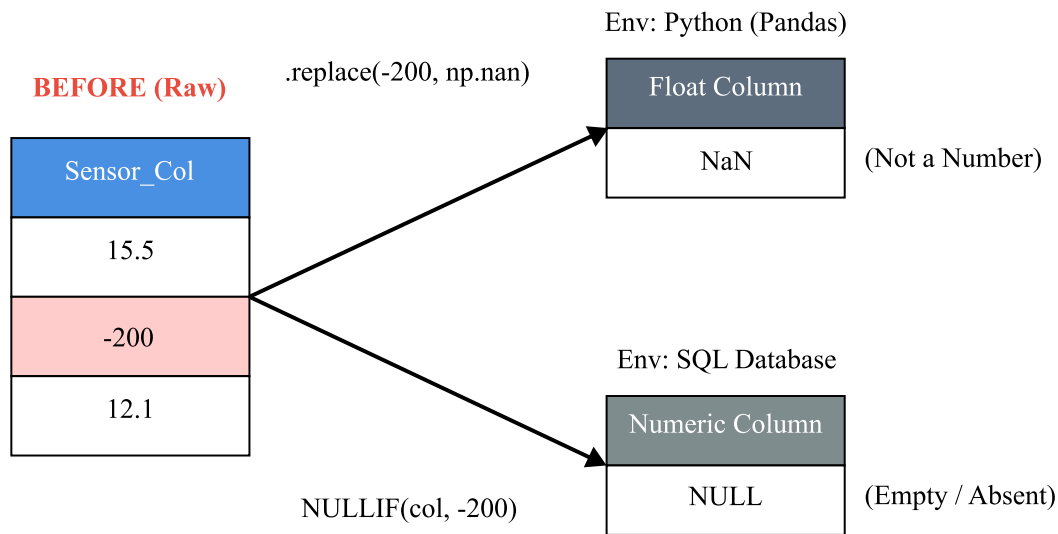
SQL: Parse timestamp (MySQL)

```

1 SELECT
2   'Date', 'Time',
3   STR_TO_DATE(CONCAT('Date', ' ', 'Time'), '%d/%m/%Y %H.%i.%s') ts
4 FROM air_quality
5 ORDER BY ts;

```

II.4. Bước 3: Chuẩn hóa missing (-200 → NaN/NULL)



Mục tiêu là đưa missing về dạng chuẩn để dễ xử lý. Ở đây, ta thay **-200** bằng NaN/NULL.

Pandas: -200 -> NaN

```
1 df[use_cols] = df[use_cols].replace(-200, np.nan)
2 df[use_cols].isna().sum()
```

Giải thích code (Pandas)

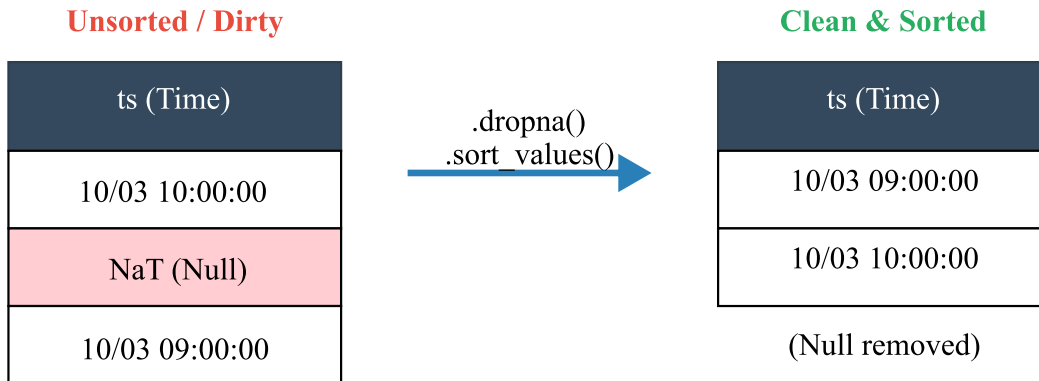
- `replace(-200, np.nan)`: đổi sentinel -200 sang NaN (missing chuẩn của Pandas).
- Sau bước này, các thao tác như `isna()`, `ffill()`, `fillna()` hoạt động đúng nghĩa với missing.
- `isna().sum()`: kiểm tra lại số missing theo từng cột sau khi chuẩn hoá.

SQL: -200 -> NULL

```
1 SELECT
2     NULLIF('CO(GT)', -200) co_gt,
3     NULLIF('NO2(GT)', -200) no2_gt,
4     NULLIF('T', -200) temp_c,
5     NULLIF('RH', -200) rh
6 FROM air_quality;
```

II.5. Bước 4: Lọc dữ liệu cột ts

Mục tiêu là đảm bảo dữ liệu có thứ tự thời gian rõ ràng. Các dòng không parse được thời gian sẽ bị loại bỏ.



Pandas: Sort theo ts và bỏ ts lỗi

```
1 df = df.dropna(subset=["ts"]).sort_values("ts").reset_index(drop=True)
2 df[["ts"] + use_cols].head()
```

Giải thích code (Pandas)

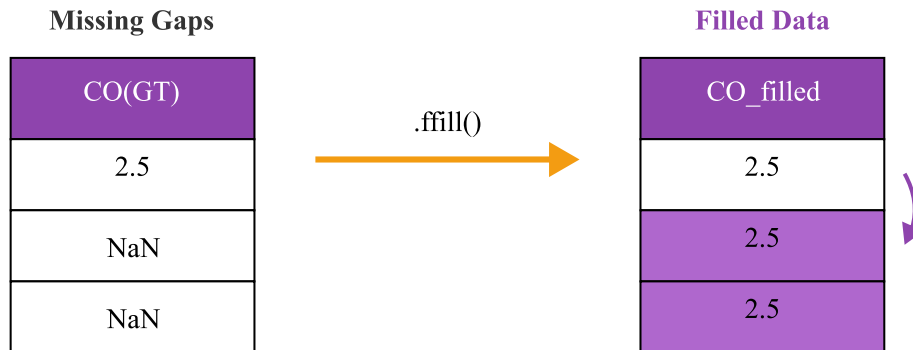
- `dropna(subset=["ts"])`: bỏ các dòng có ts bị NaT (parse lỗi).
- `sort_values("ts")`: sắp xếp theo thời gian để đúng logic time series.
- `reset_index(drop=True)`: reset index để dữ liệu "sạch" và liên tục sau khi lọc/sort.
- `df[["ts"] + use_cols].head()`: kiểm tra nhanh vài dòng đầu sau khi chuẩn hoá thứ tự thời gian.

SQL: Clean + sort theo ts

```
1 DROP VIEW IF EXISTS air_quality_clean;
2 CREATE VIEW air_quality_clean AS
3 SELECT
4     STR_TO_DATE('Time', '%Y-%m-%d %H:%i:%s') AS ts,
5     NULLIF('CO(GT)', -200) co_gt,
6     NULLIF('NO2(GT)', -200) no2_gt,
7     NULLIF('T', -200) temp_c,
8     NULLIF('RH', -200) rh
9 FROM airquality_uci_simple
10 WHERE STR_TO_DATE('Time', '%Y-%m-%d %H:%i:%s') IS NOT NULL;
11
12 SELECT * FROM air_quality_clean ORDER BY ts;
```


II.6. Bước 5: Điền missing đơn giản (impute)

Mục tiêu là tạo phiên bản dữ liệu ít thiếu hơn để phục vụ phân tích. Trong Pandas, ta dùng **forward-fill**. Trong SQL, ta minh họa bằng **điền trung bình** để giữ câu lệnh đơn giản.



Pandas: Forward-fill

```
1 for c in use_cols:
2     df[c + "_filled"] = df[c].ffill()
3
4 df[["ts"] + [c + "_filled" for c in use_cols]].head()
```

Giải thích code (Pandas)

- Duyệt từng cột trong `use_cols` để impute đồng bộ.
- `ffill()` (forward-fill): điền NaN bằng **giá trị hợp lệ gần nhất trước đó** theo thứ tự thời gian.
- Tạo cột mới `*_filled` để **giữ lại cột gốc**, giúp so sánh trước/sau.
- `head()`: kiểm tra nhanh các cột sau khi điền missing.

SQL: Impute mean

```
1 DROP VIEW IF EXISTS air_quality_filled;
2 CREATE VIEW air_quality_filled AS
3 SELECT a.ts,
4         COALESCE(a.co_gt,s.mu_co)    co_gt_filled,
5         COALESCE(a.no2_gt,s.mu_no2)  no2_gt_filled,
6         COALESCE(a.temp_c,s.mu_t)    temp_c_filled,
7         COALESCE(a.rh,s.mu_rh)       rh_filled
8 FROM air_quality_clean a
9 CROSS JOIN (
10    SELECT AVG(co_gt) mu_co, AVG(no2_gt) mu_no2, AVG(temp_c) mu_t, AVG(rh)
11           mu_rh
12    FROM air_quality_clean
13 ) s;
14 SELECT * FROM air_quality_filled ORDER BY ts;
```

II.7. Bước 6: Xác định outlier

Mục tiêu là đánh dấu các giá trị bất thường để kiểm tra. Ở đây dùng quy tắc đơn giản: gán cờ nếu CO(GT) vượt một ngưỡng cố định (ví dụ 5).

CO(GT)	is_outlier
2.1	0
15.5	1
4.8	0

if CO > 5.0:
then 1
else 0

Pandas: Flag outlier theo ngưỡng

```
1 THRESH = 5.0 # ngưỡng minh họa
2
3 df["is_outlier_co"] = (df["CO(GT)_filled"] > THRESH).astype(int)
4 df[["ts", "CO(GT)_filled", "is_outlier_co"]].tail()
```

Giải thích code (Pandas)

- THRESH: ngưỡng minh họa để phát hiện outlier (ở đây chỉ để demo ý tưởng).
- (df["CO(GT)_filled"] > THRESH) tạo boolean True/False cho từng dòng.
- astype(int) chuyển boolean thành 1/0 để tiện đếm, lọc và báo cáo.
- tail(): xem vài dòng cuối để kiểm tra cờ outlier có hoạt động đúng.

SQL: Flag outlier theo ngưỡng

```
1 SELECT
2   *,
3   (co_gt_filled > 5.0) AS is_outlier_co
4 FROM air_quality_filled
5 ORDER BY ts;
```



Các bạn có thể tham khảo 2 đoạn code sẵn ở đây:

- Pandas code: [link](#)
- Sql script: [link](#)



Pandas & SQL Windows Function

Nguyễn Thế Hào và Đinh Quang Vinh

I. Lý thuyết

📖 Window Function là gì?

Trong xử lý dữ liệu, window function là một kỹ thuật tạo đặc trưng bằng cách tính toán trên một cửa sổ các dòng liên quan đến dòng hiện tại, nhưng không làm mất từng dòng dữ liệu.

Điểm khác với phép gom nhóm:

- Gom nhóm (group aggregation): thu gọn dữ liệu, số dòng giảm (ví dụ: trung bình theo nhóm).
- Window (analytic): giữ nguyên số dòng, nhưng tạo thêm cột kết quả chạy theo nhóm và theo thứ tự (ví dụ: giá trị trước đó, trung bình trượt, thứ hạng).

Vì giữ nguyên từng dòng, window functions đặc biệt hữu ích cho dữ liệu theo thời gian và dữ liệu cần so sánh giữa các dòng liên kề.

II. Case study: Stocks (Vega Datasets)

Bộ dữ liệu Stocks

Dataset Stocks có cấu trúc gọn nên phù hợp để minh họa window functions.

- Cột nhóm: `symbol`
- Cột thời gian: `date`
- Giá trị số: `price`

Mục tiêu: minh họa `row number`, `lag/delta`, `moving average`, `running sum`, và `top-k` theo năm bằng cả Pandas và SQL.

Các bạn có thể tải dataset ở đây: [link](#)

II.1. Bước 1: Row number theo symbol

Mục tiêu là đánh số thứ tự theo thời gian trong mỗi symbol.

symbol	date ▼	row_num
AAPL	2023-01-01	1
AAPL	2023-01-02	2
AAPL	2023-01-03	3

Partition: AAPL

Partition: GOOGL

GOOGL	2023-01-01	1
GOOGL	2023-01-03	2

Pandas

```
1 dfw = df.sort_values(["symbol", "date"]).copy()
2 dfw["row_num"] = dfw.groupby("symbol").cumcount() + 1
```

Giải thích (Pandas)

- `sort_values(["symbol", "date"])`: sắp xếp dữ liệu đi đúng theo thời gian.
- `groupby("symbol")`: tách dữ liệu theo từng mã cổ phiếu; các phép tính phía sau chạy **riêng trong từng nhóm**.
- `cumcount()+1`: đếm số dòng trong mỗi nhóm theo thứ tự hiện tại (bắt đầu từ 0), nên cộng 1 để ra 1,2,3,...
- Kết quả giữ nguyên số dòng của `dfw`, chỉ thêm cột `row_num` làm chỉ số thời gian trong từng symbol.

SQL

```
1 SELECT
2     symbol, date, price,
3     ROW_NUMBER() OVER (PARTITION BY symbol ORDER BY date) AS row_num
4 FROM stocks
5 ORDER BY symbol, date;
```

II.2. Bước 2: Lag và delta

Mục tiêu là lấy giá kỳ trước và tính chênh lệch theo kỳ trong mỗi `symbol`.

date ▼	price	prev_price (LAG)	delta
01-01	100	NULL	-
01-02	105	100	+5
01-03	102	105	-3

Pandas

```
1 dfw["prev_price"] = dfw.groupby("symbol")["price"].shift(1)
2 dfw["delta"] = dfw["price"] - dfw["prev_price"]
```

Giải thích (Pandas)

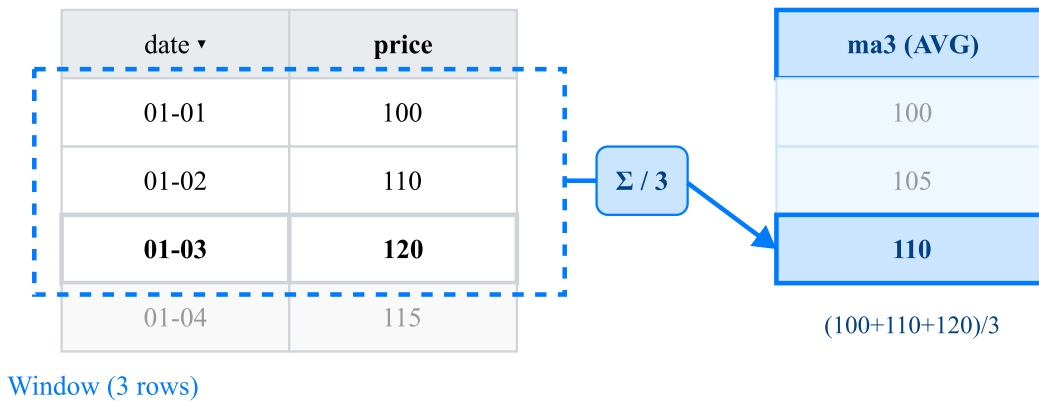
- `groupby("symbol")["price"]`: lấy cột `price` theo từng nhóm `symbol`.
- `shift(1)`: dịch xuống 1 dòng trong từng nhóm, nên mỗi dòng nhận **giá của kỳ trước** (theo `date`). Dòng đầu tiên mỗi nhóm không có kỳ trước nên `prev_price` là `NaN`.
- `delta = price - prev_price`: chênh lệch giữa hiện tại và kỳ trước (dương = tăng, âm = giảm).
- Đây là mẫu window cơ bản: so sánh “dòng hiện tại” với “dòng liền trước” trong cùng nhóm mà không cần gộp dữ liệu.

SQL

```
1 SELECT
2   symbol, date, price,
3   LAG(price) OVER (PARTITION BY symbol ORDER BY date) AS prev_price,
4   price - LAG(price) OVER (PARTITION BY symbol ORDER BY date) AS delta
5 FROM stocks
6 ORDER BY symbol, date;
```

II.3. Bước 3: Moving average và running sum

Mục tiêu là tạo trung bình trượt 3 k và tổng cộng dồn trong mỗi symbol.



Pandas

```
1 dfw["ma3"] = (
2     dfw.groupby("symbol")["price"]
3         .rolling(3, min_periods=1)
4         .mean()
5         .reset_index(level=0, drop=True)
6 )
7
8 dfw["running_sum"] = dfw.groupby("symbol")["price"].cumsum()
```

Giải thích (Pandas)

- `rolling(3, min_periods=1)`: tạo cửa sổ 3 dòng gần nhất trong từng symbol (theo date). Các dòng đầu chưa đủ 3 dòng vẫn được tính nhờ `min_periods=1`.
- `mean()`: tính trung bình trên cửa sổ, tạo cột `ma3` (moving average 3 dòng) để làm mượt biến động ngắn hạn.
- `reset_index(level=0, drop=True)`: đưa kết quả rolling về cùng index với `dfw` để gán cột trực tiếp.
- `cumsum()`: tổng cộng dồn trong từng symbol (tổng từ đầu chuỗi đến dòng hiện tại).

SQL

```
1 SELECT
2     symbol, date, price,
3     AVG(price) OVER (
4         PARTITION BY symbol ORDER BY date
5         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
6     ) AS ma3,
7     SUM(price) OVER (
8         PARTITION BY symbol ORDER BY date
```

```

9      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
10    ) AS running_sum
11 FROM stocks
12 ORDER BY symbol, date;

```

II.4. Bước 4: Top-3 (RANK)

Mục tiêu là xếp hạng theo năm trong từng symbol và lấy top-3 tháng có giá cao nhất.

symbol	year	date	price ▾	rnk
AAPL	2023	12-20	150	1
AAPL	2023	08-15	145	2
AAPL	2023	03-10	140	3
AAPL	2023	01-01	100	4

Partition: AAPL, 2023

Giữ lại
(Top 3 cao nhất)

Loại bỏ
(Rank > 3)

Pandas

```

1 dfw["year"] = dfw["date"].dt.year
2 dfw["rank_year"] = dfw.groupby(["symbol", "year"])["price"].rank(
3     ascending=False, method="min"
4 )
5
6 top3 = dfw[dfw["rank_year"] <= 3].sort_values(["symbol", "year", "
7     rank_year", "date"])

```

Giải thích (Pandas)

- `year = dfw["date"].dt.year`: tách năm từ `date` để dùng cho việc nhóm theo năm.
- `groupby(["symbol", "year"])`: mỗi (mã, năm) là một nhóm riêng, xếp hạng độc lập.
- `rank(ascending=False, method="min")`: xếp hạng theo `price` giảm dần (hạng 1 là giá cao nhất). Nếu bị trùng giá, `method="min"` cho tất cả cùng nhận hạng nhỏ nhất (giống RANK trong SQL).
- `rank_year <= 3`: lọc top-3 theo từng (mã, năm) — mẫu thường gặp: **tính rank trong nhóm rồi lọc theo rank**.

SQL

```
1 WITH ranked AS (  
2     SELECT  
3         symbol, date, price,  
4         YEAR(date) AS year,  
5         RANK() OVER (  
6             PARTITION BY symbol, YEAR(date)  
7             ORDER BY price DESC  
8         ) AS rnk  
9     FROM stocks  
10 )  
11 SELECT *  
12 FROM ranked  
13 WHERE rnk <= 3  
14 ORDER BY symbol, year, rnk, date;
```



Các bạn có thể tham khảo 2 đoạn code sẵn ở đây:

- Pandas code: [link](#)
- Sql script: [link](#)